

Question 6 – Stage 1

The triangle with vertices $A(1, 3)$, $B(-2, 4)$ and $C(3, -5)$ is not right-angled.

Changing only one of these six coordinates can result in ABC being the vertices of a right-angled triangle.

Find (to 3d.p.) the area of the smallest right-angled triangle that can be created in this way.

Working

```
In [1]: from itertools import permutations

points = [(1,3),(-2,4),(3,-5)]

def vectorBetweenPoints(p1,p2):
    return (p2[0] - p1[0],p2[1] - p1[1])

def gradientOfTwoPoints(p1,p2):
    dp = vectorBetweenPoints(p1,p2)
    return dp[1]/dp[0]

def findLineX(gradient,offset,desiredY):
    return (desiredY-offset)/gradient

def findLineY(gradient,offset,desiredX):
    return gradient * desiredX + offset

def triangleAreaFromPoints(p1,p2,p3):
    x1,y1 = p1
    x2,y2 = p2
    x3,y3 = p3
    return 0.5*abs(x1*(y2-y3)+x2*(y3-y1)+x3*(y1-y2))

def quadraticSolve(a,b,c):
    discriminant = b**2 - 4 * a * c
    if discriminant < 0 :
        return "Imaginary"
    positive = (-b + discriminant**0.5) / (2*a)
    negative = (-b - discriminant**0.5) / (2*a)
    return [positive,negative]

areas = []

# Move point not at the right angle
for combination in permutations(points,len(points)):
    # First of pairing is the location of the right angle, 2nd is the reference, 3rd is the moving.
    linePoint,refPoint,movedPoint = combination
    solveLineGradient = -1/gradientOfTwoPoints(linePoint,refPoint)
    solveLineOffset = linePoint[1] - findLineY(solveLineGradient,0,linePoint[0])

    # move X
    movedPoint = (findLineX(solveLineGradient,solveLineOffset,movedPoint[1]),movedPoint[1])
    areas.append(triangleAreaFromPoints(linePoint,refPoint,movedPoint))

    # move Y
    movedPoint = (movedPoint[0],findLineY(solveLineGradient,solveLineOffset,movedPoint[0]))
    areas.append(triangleAreaFromPoints(linePoint,refPoint,movedPoint))

# Move point at the right angle
for point in points:
    # move X
    pointsTemp = points.copy()
    pointsTemp.remove(point)
    x1,y1 = point
    x2,y2 = pointsTemp[0]
    x3,y3 = pointsTemp[1]

    b = -1 * (x3 + x2)
    c = x3 * x2 + (y2-y1) * (y3-y1)

    solve = quadraticSolve(1,b,c)
    if solve != "Imaginary":
        for xCoord in solve:
            movedPoint = (xCoord,y1)
            areas.append(triangleAreaFromPoints(linePoint,refPoint,movedPoint))

    # move Y
    b = -1 * y3 + y2
    c = y3 * y2 + (x2-x1) * (x3-x1)

    solve = quadraticSolve(1,b,c)
    if solve != "Imaginary":
        for yCoord in solve:
            movedPoint = (x1,yCoord)
            areas.append(triangleAreaFromPoints(linePoint,refPoint,movedPoint))

areas.sort()
print(f"Min Area: {round(areas[0],3)}")
```

Min Area: 8.237